

¿Puede haber ciclos en una red neuronal?

¿Puede haber ciclos en una red neuronal?

Si es una red feedforward, no se puede. Sí en RNNs y arquitecturas recurrentes.

Explicar la estructura del perceptrón

Explicar la estructura del perceptrón

Se cuenta con los inputs (escalares), un peso de ese input y un bias (opcionalmente).

Se hace la suma de los inputs multiplicados por sus respectivos pesos. Finalmente, se aplica una función de activación.

¿Qué es un MLP? ¿Cómo se organiza?

¿Qué es un MLP? ¿Cómo se organiza?

Un MLP es un perceptrón multicapa.

Un MLP cuenta con todas capas fully connected (FC). Tiene una input layer, hidden layer(s) y una output layer.

Definir shallow y deep network

Definir shallow y deep network

Una shallow network tiene pocas capas ocultas (es más ancha).

Una deep network tiene muchas capas ocultas (es más profunda).

¿Qué es un tensor? ¿Qué ventajas tiene?

¿Qué es un tensor? ¿Qué ventajas tiene?

Un tensor es un arreglo multidimensional de escalares.

Sirve para datos de alta dimensionalidad (como imágenes o audios), es eficiente para GPU, y es una representación fácil de inputs, outputs, weights y biases.

Definir forward propagation y su procedimiento.

Definir forward propagation y su procedimiento.

Forward propagation computa el output para un input.

Se hacen los cálculos a través de cada capa en el sentido de IL/HL/OL para conseguir una predicción $\hat{y}$. A medida que se avanza, se guardan los parámetros para usar en backward propagation.

Definir backward propagation y su procedimiento. ¿Qué ventaja y desventaja tiene?

Definir backward propagation y su procedimiento. ¿Qué ventaja y desventaja tiene?

Es el proceso de actualizar los pesos y biases usando el gradiente de la función loss. Se busca minimizar la función loss en el training set. Va en sentido contrario a forward propagation.

Primero, se calcula la función loss definida por $\hat{y} - y$ a grandes rasgos. Luego, se calculan los gradientes usando la regla de la cadena. Luego, se aplica el learning rate LR al gradiente. Finalmente, se actualizan los pesos y biases yendo en sentido contrario: OL/HL/IL.

Ventaja: es eficiente, ya que una observación del dataset requiere una pasada forward y una pasada backward.

Desventaja: puede sufrir vanishing/exploding gradients, lo que dificulta entrenar redes profundas.

Nombrar los dos tipos de capas.

Nombrar los dos tipos de capas.

Capas entrenables: sus pesos y biases se modifican en backpropagation.

Capas no entrenables: sus pesos y biases quedan fijos en training para mantener las features aprendidas previamente.

¿Qué es la vectorización y para qué sirve? ¿Cuáles son sus ventajas? ¿Cómo queda el cálculo de una neurona en una hidden layer?

¿Qué es la vectorización y para qué sirve? ¿Cuáles son sus ventajas? ¿Cómo queda el cálculo de una neurona en una hidden layer?

Es una práctica común para almacenar pesos, inputs y biases en vectores.

Sirve para simplificar las operaciones y acelerar los tiempos.

Las ventajas que trae son la escalabilidad, la claridad, la capacidad de procesamiento en batch, performance y paralelismo.

$$\phi(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

Donde ϕ es la función de activación, $\mathbf{W}$ el vector de pesos, $\mathbf{x}$ el vector de inputs y $\mathbf{b}$ el vector de biases.

¿Qué es y para qué sirve la función loss?

¿Qué es y para qué sirve la función loss?

Es una función que mide la discrepancia entre $\hat{y}$ e y .

Sirve para guiar el entrenamiento de la red.

¿Para qué sirven los grafos computacionales?

¿Para qué sirven los grafos computacionales?

Para reutilizar cálculos y ayudar con la eficiencia del backpropagation. Las librerías lo usan mucho.

¿Para qué sirven y por qué son importantes las funciones de activación?

¿Para qué sirven y por qué son importantes las funciones de activación?

Sirven para aprender mapeos no lineales y para capturar estructuras complejas de los datos.

De no tenerlas, la red se colapsaría a una única capa con la combinación lineal.

Nombrar y describir las funciones de activación lineales.
¿Qué ventajas y qué desventajas tiene?

Nombrar y describir la función de activación lineal. ¿Qué ventajas y qué desventajas tiene?

Función de identidad: el output es el mismo que el input. $f(x) = x$.

Ventajas: simples y eficientes, son fáciles de interpretar.

Desventajas: limitadas con datos con patrones complejos, colapsa la red. Generalmente se usa en la capa de output.

Nombrar y describir las funciones escalonadas. ¿Qué ventajas y qué desventajas tienen?

Nombrar y describir las funciones escalonadas. ¿Qué ventajas y qué desventajas tienen?

Heaviside step: cuando el valor está del lado izquierdo a un umbral tiene un valor. Cuando está del lado derecho tiene otro.

Función de signo: es una función heaviside step pero con -1 para valores negativos y 1 para valores positivos.

Ventajas: son simples y fáciles de implementar y las salidas binarias convienen para problemas de clasificación.

Desventajas: no es diferenciable en el threshold, es sensible a pequeños cambios en el input y no posee información de gradiente para aprender.

Nombrar y describir las funciones lineales por partes.
¿Qué ventajas y qué desventajas tienen?

Nombrar y describir las funciones lineales por partes. ¿Qué ventajas y qué desventajas tienen?

ReLU: si el input es negativo, el output es cero. Si el input es positivo, es la función identidad. $f(x) = \max(0, x)$

Leaky ReLU: el resultado para los valores negativos cambia. Es igual a ReLU solo que la parte negativa está inclinada.

Shifted ReLU: es ReLU pero con otro threshold.

Ventajas (ReLU): simple y eficiente, introduce no linealidad en la red, ayuda con el problema de vanishing gradient.

Desventajas (ReLU): las neuronas con el input negativo pueden desactivarse.

Ventajas (var. ReLU): threshold variable, la neurona no puede morir, mismos beneficios que ReLU.

Desventajas (var. ReLU): más hiperparámetros.

Nombrar y describir las funciones suaves. ¿Qué ventajas y qué desventajas tienen?

Nombrar y describir las funciones suaves. ¿Qué ventajas y qué desventajas tienen?

Softplus: es la versión suave de la ReLU. $f(x) = \ln(1 + e^x)$.

ELU: es la versión suave de la Shifted ReLU.

Sigmoide: es una curva con forma de S, con su output acotado entre 0 y 1.

Tanh: es igual que la sigmoide, con su output acotado entre -1 y 1.

Swish: es una combinación entre ReLU y Sigmoide.

Ventajas: son suaves y diferenciables. La sigmoide sirve para clasificación, tanh para convergencia, swish usa self-gating y ELU evita que la neurona muera.

Desventajas: son costosas y sufren del vanishing gradient.

Describir el problema de Vanishing Gradient y Dying Neuron.

Describir el problema de Vanishing Gradient y Dying Neuron.

Vanishing Gradient: cuando los gradientes se hacen muy pequeños durante backpropagation a través de las capas.

Dying Neuron: cuando las neuronas siempre están inactivas y sus pesos no se actualizan dado que su gradiente es cero.

Explicar la función Softmax y Log-Softmax.

Explicar la función Softmax y Log-Softmax.

Softmax: es una operación que se aplica únicamente y en simultáneo a todas las neuronas en la capa de output. Sirve para normalizar las salidas a una distribución de probabilidad.

Log-Softmax: es hacer el logaritmo de la función SoftMax. Sirve para aportar estabilidad a los números.

¿Qué es un gradiente? ¿Para qué sirve?

¿Qué es un gradiente? ¿Para qué sirve?

Es un vector de derivadas parciales que indica la dirección de máximo aumento de la función loss.

Sirve para minimizar la función loss, entonces se avanza en el sentido contrario al gradiente.

¿Qué es el error landscape? ¿Qué problema sufren?

¿Qué es el error landscape? ¿Qué problema sufren?

El **Error Landscape** es la geometría de la función loss. Puede haber posibles representaciones de esta geometría: máximos y mínimos, mesetas y sillas de montar.

Las representaciones suelen dificultar la optimización.

¿Qué es el learning rate? ¿Para qué sirve? ¿Cómo responde un optimizador ante ciertos valores del LR?

¿Qué es el learning rate? ¿Para qué sirve? ¿Cómo responde un optimizador ante ciertos valores del LR?

Es un hiperparámetro denotado con ϵ .

Controla el tamaño del paso para actualizar los pesos.

Si se tiene un LR grande, el modelo diverge. Si se tiene un LR chico, el modelo aprende lentamente. La trayectoria en el espacio de parámetros depende de este hiperparámetro.

¿Qué es un schedule? ¿Qué tipos de schedules hay?

¿Qué es un schedule? ¿Qué tipos de schedules hay?

Es el ajuste del LR en el tiempo para que no sea constante siempre.

1. **Decaimiento exponencial.**
2. **Decaimiento exponencial retrasado.** El LR constante y luego decrece.
3. **Decaimiento basado en el error.** Disminuye el LR siempre y cuando baje el error.

Explicar tres tipos de descenso de gradiente. ¿Qué ventajas y desventajas tienen?

Explicar tres tipos de descenso de gradiente. ¿Qué ventajas y desventajas tienen?

1. **Batch (BGD)**: se utiliza todo el dataset de una. Es estable y preciso, pero costoso y lento.
2. **Estocástico (SGD)**: se usa una observación por paso. Es rápido y simple, pero inestable y ruidoso.
3. **Mini-Batch**: usa pequeños grupos de observaciones por paso. Es barato y preciso, pero el resultado depende del tamaño del batch.

¿Qué es el momentum? ¿Para qué se usa?

¿Qué es el momentum? ¿Para qué se usa?

Es un hiperparámetro que agrega inercia al update de los pesos.

Sirve para suavizar la trayectoria y para escapar de las mesetas.

Describir tres optimizadores de LR adaptativos. ¿Qué ventajas y desventajas tienen?

Describir tres optimizadores de LR adaptativos. ¿Qué ventajas y desventajas tienen?

1. **AdaGrad**: escala LR inversamente a la raíz de la suma de los gradientes acumulados. Este funciona bien con features esparsas, pero el LR decae demasiado.
2. **Adadelta / RMSprop**: usa la suma decay. Este converge más rápido pero introduce otro hiperparámetro **decay**.
3. **ADAM**: combina promedios de primer y segundo momento (el gradiente y su cuadrado). Es muy eficiente y común, pero suma dos hiperparámetros: β_1 y β_2 .

Describir tres métodos de búsqueda de hiperparámetros.

Describir tres métodos de búsqueda de hiperparámetros.

1. **Grid Search:** se prueban todas las combinaciones posibles de hiperparámetros. Es un proceso simple pero muy costoso.
2. **Random Search:** se toman combinaciones de hiperparámetros al azar. Es más eficiente que Grid Search, pero no garantiza el óptimo.
3. **Optimización Bayesiana:** se hacen evaluaciones previas para guiar la búsqueda de hiperparámetros. Es un método eficiente y adaptativo pero es difícil de implementar.

Nombrar seis métodos de regularización.

Nombrar seis métodos de regularización.

1. **L1**: penaliza valores absolutos y genera modelos simples y esparsos.
2. **L2**: penaliza cuadrados y genera pesos más suaves y distribuidos.
3. **Dropout**: desactiva un porcentaje de las neuronas aleatoriamente. Esto mejora la generalización.
4. **Data Augmentation**.
5. **Noise Injection**.
6. **Early Stopping**.

¿Qué es y para qué sirve una convolución? Explicar sus componentes.

¿Qué es y para qué sirve una convolución? Explicar sus componentes.

Una **convolución** es una operación matemática que toma dos funciones o dos matrices y genera una tercera a partir de ellas. Sirve para medir cómo cambia una función cuando interactúa con otra. Se nota con $(f * g)(x)$.

f es el input y g es el kernel. El input suele ser una imagen y el kernel es una matriz chica. El resultado es el feature map, que son algunas características que capturó el kernel.

¿Por qué son convenientes las convoluciones multidimensionales?

¿Por qué son convenientes las convoluciones multidimensionales?

Se pueden hacer convoluciones entre dos tensores, lo que permite hacer cálculos en más de una dimensión a la vez.

Además, se usan menos parámetros y los pesos se comparten (porque el mismo kernel se aplica en toda la imagen). Tiene muchos menos parámetros que una capa fully-connected.

¿Qué es un filtro? ¿Para qué sirve?

¿Qué es un filtro? ¿Para qué sirve?

Un filtro está compuesto por un kernel por cada canal de entrada y produce un feature map.

Sirve para producir más de un feature map.

Explicar padding, stride y pooling.

Explicar padding, stride y pooling.

- **Padding:** agrega bordes de píxeles extras para mantener el tamaño espacial de la salida y no perder información de los bordes.
- **Stride:** controla el desplazamiento del kernel por sobre la imagen. Afecta al tamaño del output y la cantidad de downsampling.
- **Pooling:** reduce la dimensión espacial de la imagen (downsampling). Se reemplaza una porción de la imagen por un resumen estadístico de sus valores. Este proceso reduce el cálculo de las capas siguientes. Se puede hacer max pooling o average pooling.

Explicar la arquitectura de una CNN.

Explicar la arquitectura de una CNN.

1. Capas iniciales: aprenden bordes o direcciones.
2. Capas intermedias: aprenden texturas o formas simples.
3. Capas finales: aprenden partes de objetos o patrones complejos.

¿Qué es el campo receptivo? ¿Cómo cambia a medida que se acerca a las capas finales?

¿Qué es el campo receptivo? ¿Cómo cambia a medida que se acerca a las capas finales?

Es la zona del input que ve una neurona.

En las capas bajas, los campos pequeños detectan detalles simples. En las capas altas, los campos grandes detectan patrones globales.

Describir las tres tareas de reconocimiento.

Describir las tres tareas de reconocimiento.

1. **Clasificación:** hay una etiqueta por imagen.
2. **Clasificación con localización:** etiqueta y posición de un único objeto por imagen.
3. **Detección:** reconocimiento de múltiples objetos y sus posiciones en la imagen.

Describir la arquitectura LeNet.

Describir la arquitectura LeNet.

LeNet es una (de las primeras) CNN que se usa para reconocimiento de dígitos. Muestra cómo pasar de una imagen grande a varias representaciones más chicas, usando el kernel para extraerle características.

Describir la arquitectura AlexNet y sus hiperparámetros.

Describir la arquitectura AlexNet y sus hiperparámetros.

AlexNet una CNN que aprovecha las GPUs. Tiene las siguientes características:

- Tiene una función de activación ReLU para evitar saturación y apurar el training.
- Usa dropout para reducir el overfitting.
- Usa aumentación de datos para generalizar mejor.
- Hace preprocesamiento para mejorar la estabilidad y el rendimiento normalizando las entradas.

Los hiperparámetros que tiene son: Mini-batch, Momentum, Decay, LR.

Describir la arquitectura VGGNet.

Describir la arquitectura VGGNet.

Es una CNN que estudia cómo la profundidad afecta a la performance. Tiene las siguientes características:

- Usa kernels chicos para reducir los parámetros.
- Usa la optimización L2 para weight decay.
- No tiene local response normalization.
- Preprocesamiento: resta la media RGB del training set.

Describir la arquitectura GoogLeNet con Inception.

Describir la arquitectura GoogLeNet con Inception.

Es una CNN más rápida y eficiente que AlexNet.

- Tiene capas de módulos Inception. Usa convoluciones 1x1 para reducir dimensionalidad y luego concatena los outputs de distintos tamaños de kernel.
- Se hace average pooling para evitar overfitting.
- Se usan funciones de loss intermedias para evitar vanishing gradient.

Describir la arquitectura ResNet.

Describir la arquitectura ResNet.

Es una CNN que usa conexiones residuales (skip connections) para que los gradientes fluyan directamente. Se usan bloques residuales apilados que permiten que la red sea más profunda pero sin bajar su rendimiento. Entre más profunda es ResNet, mejor su representación.

Describir la arquitectura CAM.

Describir la arquitectura CAM.

Muestra qué partes de la imagen son las más importantes para predecir una clase. La red aprende los pesos por clase para cada feature map. Se hace un global average pooling antes de aplicar softmax.

Describir la arquitectura Grad-CAM.

Describir la arquitectura Grad-CAM.

Es una variante de CAM. Usa gradiente del score de clase respecto al último feature map. Resalta las zonas con mayor influencia en la decisión final. Permite interpretar CNNs y muestran qué ven esas redes al clasificar.

¿Qué es un encoder? ¿Y un decoder? ¿Por qué querríamos usarlos?

¿Qué es un encoder? ¿Y un decoder? ¿Por qué querríamos usarlos?

Un **encoder** toma una imagen y la convierte a una representación compacta, que es un vector de característica.

Un **decoder** toma la representación compacta y hace la reconstrucción de la imagen original o genera otra salida visual.

Sirven por si queremos un output más sofisticado que no sea un número o una etiqueta cuando le paso una imagen.

¿Qué es la interpolación? Nombrar dos tipos.

¿Qué es la interpolación? Nombrar dos tipos.

Es un método de upscaling (aumentar la resolución) de una imagen generando nuevos píxeles en base a la imagen pasada por parámetro. Es una fórmula fija.

- **Vecino más Cercano:** cada nuevo píxel toma el valor del píxel existente más cercano. Es un método simple y rápido.
- **Bilineal:** cada nuevo píxel es un promedio ponderado de los cuatro píxeles vecinos, para obtener resultados más suaves.

¿Qué es una convolución transpuesta? ¿Qué hace?

¿Qué es una convolución transpuesta? ¿Qué hace?

Hace el proceso inverso a la convolución, es decir, aprende el upsampling.

Con el stride y el padding, la convolución transpuesta devuelve una imagen expandida (lo opuesto a una convolución normal). Esta operación permite a la red aprender a generar los píxeles.

¿Qué se busca lograr con la segmentación de imágenes?

¿Qué se busca lograr con la segmentación de imágenes?

Se busca predecir una etiqueta por pixel.

Describir los cinco tipos de segmentación de imágenes.

Describir los cinco modelos de reconocimiento visual.

1. **U-Net**: tiene una arquitectura encoder-decoder con forma de $\tilde{U}$. En la última capa se hace una convolución de 1×1 para luego pasar el output por softmax.
2. **YOLO**: divide la imagen en una grilla y predice la clase y atributos de cada celda. Optimiza usando una loss que combina el tamaño de las bounding box, confianza y probabilidad de clase.
3. **R-CNN**: genera regiones candidatas y extrae características. Luego, usa una red para clasificar el objeto en cada región.
4. **SAM**: permite segmentar cualquier objeto de una imagen con un prompt.
5. **FAN**: detecta puntos de interés en las caras usando convoluciones y capas FC. Predice heatmaps que indican la probabilidad de cada punto.

¿Qué es un autoencoder? ¿Qué arquitectura tiene?

¿Qué es un autoencoder? ¿Qué arquitectura tiene?

Un **autoencoder** es una red neuronal que busca aprender una representación comprimida de los datos. Toma una entrada x y busca reconstruirla como $\hat{x}$.

Tiene un encoder que reduce la dimensionalidad de un input y consigue un código latente, y tiene un decoder para reconstruir la entrada original a partir del código latente.

Describir los variational autoencoders (VAEs).

Describir los variational autoencoders (VAEs).

Funcionan como una extensión de los autoencoders clásicos usando probabilidades.

El encoder produce las distribuciones (media y varianza) sobre el espacio latente, que es donde se radica el código latente. El modelo hace que las distribuciones se parezcan a una distribución normal. Eso sirve para que se puedan muestrear puntos del espacio latente y generar nuevos datos gracias al decoder.

Describir los generative adversarial networks (GANs).

Describir los generative adversarial networks (GANs).

Tienen dos redes entrenadas en conjunto: un generador que produce muestras falsas y un discriminador que estima una probabilidad de que esa muestra sea real.

En el entrenamiento, el generador y el discriminador compiten: el generador intenta engañar al discriminador, mientras que el discriminador intenta adivinar si la muestra es verdadera o falsa (generada).

¿Qué es una red neuronal recurrente? ¿Por qué se necesitan?

¿Qué es una red neuronal recurrente? ¿Por qué se necesitan?

Es una red con ciclos en su estructura para recordar estados previos y poder procesar datos secuenciales.

Se necesitan porque las redes feedforward solamente procesan información hacia adelante, no pueden procesar datos secuenciales ni pueden recordar estados previos.

¿Cómo es el entrenamiento de una RNN?

¿Cómo es el entrenamiento de una RNN?

Se usa el mecanismo de backpropagation through time (BPTT), donde los gradientes se propagan para atrás a través de todas las etapas de la secuencia. Si la secuencia es muy larga, se usa BPTT truncado para que no sea tan costoso.

Nombrar las ventajas y desventajas de una RNN.

Nombrar las ventajas y desventajas de una RNN.

Ventajas:

- Manejan secuencias de longitud variable.
- Tienen memoria interna.
- Comparten pesos.
- Pueden combinarse con CNNs.

Desventajas:

- Entrenamiento lento.
- Propenso a vanishing y exploding gradient.
- No capturan las dependencias largas.

¿Qué es una LSTM? ¿Qué componentes tiene?

¿Qué es una LSTM? ¿Qué componentes tiene?

Es una RNN diseñada para resolver el vanishing/exploding gradient y capturar dependencias. Usa el producto de Hadamard $\odot$ para conservar información útil a lo largo del tiempo.

Tiene un funcionamiento de tres puertas:

- Puerta de entrada: decide qué información nueva entra a la memoria.
- Puerta de olvido: decide qué información se descarta.
- Puerta de salida: decide qué información se usa como salida.

Describir un LSTM bidireccional (BiLSTM).

Describir un LSTM bidireccional (BiLSTM).

No es más que un LSTM que procesa la información en ambas direcciones. Permite la incorporación del contexto completo, porque a veces las palabras tienen un significado particular según lo que venga después.

Nombrar las limitaciones de las RNNs.

Nombrar las limitaciones de las RNNs.

1. Operar secuencialmente con la información es un proceso lento.
2. La información del pasado se olvida con la longitud de la secuencia.

Explicar el mecanismo attention.

Explicar el mecanismo attention.

Este mecanismo permite que la red se enfoque en lo importante de la entrada. Se le asignan pesos a cada palabra (token) según su relevancia contextual y se puede procesar el contexto global, sea corta o larga la secuencia.

El procesamiento deja de ser secuencial, sino que se mira toda la secuencia al mismo tiempo.

Explicar el mecanismo self-attention.

Explicar el mecanismo self-attention.

Usa el mismo mecanismo que attention, solo con la restricción de que la secuencia se atiende a sí misma, es decir que cada token mira a todos los otros tokens de la misma oración.

¿Qué es un transformer? ¿Qué estructura tiene?

¿Qué es un transformer? ¿Qué estructura tiene?

Un **transformer** es un reemplazo de las RNNs que permite la paralelización del procesamiento de las palabras.

En la arquitectura original se usan seis encoders apilados y seis decoders apilados. Tiene embeddings posicionales para indicar el orden de las palabras.

Cada encoder tiene self-attention, una red neuronal feedforward y conexiones residuales.

Cada decoder tiene lo mismo, solo que el self-attention usa una máscara.

Describir qué es BERT.

Describir qué es BERT.

Es un modelo basado únicamente en encoders. Es un modelo bidireccional donde se analiza el contexto a la izquierda y derecha de una palabra.

En el preentrenamiento se usa Masked Language Modeling (MLM) y Next Sentence Prediction (NSP). Luego, se aplica fine-tuning para ajustar el modelo a tareas específicas.

Explicar el proceso de Vision Transformer (ViT).

Explicar el proceso de Vision Transformer (ViT).

1. La imagen se divide en patches.
2. Cada patch se convierte en un embedding.
3. Se le aplica un transformer como si fuera una secuencia de tokens.